Three Job Types Suffice for NP-Complete Unrelated-Machine Scheduling

Research manuscript · not peer reviewed
16 September 2026

Abstract

High-multiplicity scheduling represents repeated jobs by binary multiplicities. We prove that makespan feasibility on explicitly listed unrelated machines is NP-complete with at most three job types. The reduction from Balanced Partition uses a common auxiliary objective and a machine-dependent perturbation to expose exactly two feasible configurations per item. Global job counts force every machine to choose one of these configurations. Two column sums then recover both the weight and cardinality of a balanced subset from every feasible target allocation. A product rescaling gives one common deadline with polynomial encoding length. The construction improves the located four-type hardness result without asserting strong hardness or an approximation gap.

1. Introduction

The number of distinct job types can be small even when the number of jobs is large. In the high-multiplicity model, an instance records the multiplicity of each type in binary. Algorithms polynomial in the expanded number of jobs need not be polynomial in this input representation. Determining how few types can already support NP-hardness therefore identifies a basic limit of compressed scheduling algorithms.

Koutecký and Zink prove NP-hardness for four job types on unrelated machines in Theorem 4.14 of [1]. The paragraph following that theorem and Section 5.2 ask for the complexity at smaller type counts. Earlier two-length algorithms appear in [2]; its Section 5 also states an unrelated-machine extension. Our result concerns the three-type decision problem and does not require a new classification at two types.

Theorem 1. Makespan feasibility on explicitly listed unrelated machines, with positive integer processing times and binary job multiplicities, is NP-complete with at most three job types. There is a deterministic polynomial-time Karp reduction from Balanced Partition and a polynomial-time algorithm that recovers a balanced subset from every feasible allocation of the constructed instance.

The construction forces two configurations by saturating an auxiliary linear objective. This objective is used only in the proof: the target instance has the ordinary makespan constraints. A modular equation limits the configurations that attain the auxiliary bound, and a nonnegative perturbation excludes all but two of them. The difference between these two configurations depends on the source weight. Their global sums encode the two equations of Balanced Partition.

2. Decision problems

A Balanced Partition instance is an even-length list of positive binary integers $a_1, \dots, a_m$. Write $A = \sum_{i=1}^{m} a_i$. A solution is a subset $S \subseteq \{1, \dots, m\}$ such that $|S| = \frac{m}{2}$ and $\sum_{i \in S} a_i = \frac{A}{2}$. The empty list is a YES instance; an odd total is a NO instance.

This source problem is NP-hard. To reduce ordinary Partition, append as many zero weights as there are original weights, then add one to every entry. An original subset can be padded with zeros to half the new cardinality. The shift adds the same amount to both balanced sides. The resulting entries are positive, and subtracting the shift recovers the original partition equation. Related balanced-source transformations are also given in Lemma 4.8 of [1].

A target instance specifies positive processing times P_{ij} for each machine i and job type j, nonnegative multiplicities n_j, and a nonnegative common deadline T. There are at most three types. A feasible witness is a nonnegative integer count matrix c satisfying

$$\sum_i c_{ij} = n_j \quad \text{for every type } j, \quad \sum_j P_{ij} c_{ij} \le T \quad \text{for every machine } i. \tag{1}$$

Jobs are independent and nonpreemptive. Each row can be executed consecutively, so these inequalities are equivalent to a schedule of makespan at most T.

3. Construction

Assume first that the source is nonempty and A is even. Set

$$k = \max_i a_i, \quad H = 4(k+1)^2, \quad w = (k, k+1, 2k+1). \tag{2}$$

For each item $a = a_i$, define an auxiliary processing row and local capacity:

$$\begin{aligned} b_i &= (k+1)(2k+1-a), \quad s_i = a(2k+1-a), \\ t_i &= (2k+1-a, a, 0), \quad p_i = Hw + t_i, \quad C_i = Hb_i + s_i. \end{aligned} \tag{3}$$

All processing times and capacities are positive. The intended configurations are

$$u_i = (0, 2k+1-a_i, 0), \quad v_i = (a_i, 0, k+1-a_i). \tag{4}$$

Selecting v_i will mean selecting item i in the balanced subset.

To express these local capacities using one common deadline, set

$$T = \prod_{i=1}^{m} C_i, \quad P_{ij} = \left(\frac{T}{C_i}\right) p_{ij}. \tag{5}$$

Each quotient is an integer, and the resulting row inequality in Equation 1 is equivalent to $p_i \cdot c_i \le C_i$. The emitted instance has the multiplicities

$$n = \left(\frac{A}{2}, \frac{m}{2}(2k+1) - \frac{A}{2}, \frac{m}{2}(k+1) - \frac{A}{2}\right). \tag{6}$$

They are nonnegative integers because $A \le mk$.

For the empty source, output a single machine with processing time one, zero jobs and deadline zero. For an odd total, output a single machine with processing time one, one job and deadline zero. These are legal YES and NO instances, respectively. Malformed encodings map to an invalid target encoding.

4. Correctness and witness recovery

Lemma 2 (local selector). For $1 \le a_i \le k$, every feasible nonnegative integer configuration c_i satisfies $w \cdot c_i \le b_i$. Equality holds precisely at the two configurations in Equation 4.

Proof. The vector t_i is nonnegative, and $s_i \le k(k+1) < H$. Thus $w \cdot c_i \ge b_i + 1$ would imply $p_i \cdot c_i \ge H(b_i+1) > C_i$.

Consider a configuration (x, y, z) with $w \cdot (x, y, z) = b_i$. Reduction modulo $k+1$ gives $x + z = r(k+1)$ for a nonnegative integer r. Since

$$k(x+z) \le b_i \le 2k(k+1), \tag{7}$$

only $r = 0, 1, 2$ are possible. When $r = 0$, the configuration is u_i. When $r = 1$, substitution gives

$$y + z = k + 1 - a_i, \quad (x, y, z) = v_i + q(1, 1, -1), \quad 0 \le q \le k + 1 - a_i. \tag{8}$$

The perturbation cost is $s_i + q(2k+1)$, so feasibility requires $q = 0$. When $r = 2$, equality in Equation 7 forces $a_i = 1$ and $(x, y, z) = (2(k+1), 0, 0)$. Its perturbation cost is $4k(k+1) > 2k = s_i$, so it is infeasible. Finally, u_i and v_i both have auxiliary objective b_i and perturbation cost s_i, and therefore load C_i. □

Figure 1 illustrates the exclusion in the middle case. Although several integer configurations attain the auxiliary objective, the perturbation retains only the two configurations that can encode an item choice.

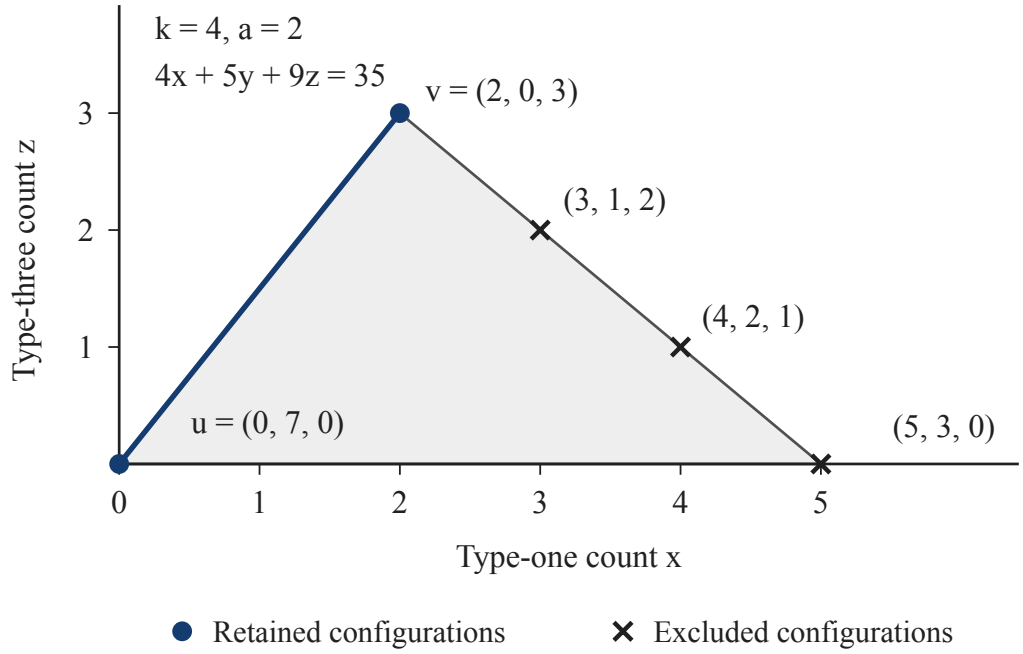


Figure 1: The complete integer fiber $4x + 5y + 9z = 35$ for $k = 4$ and $a_i = 2$, projected onto the first and third counts. Labels give all three counts (x, y, z). Filled circles are the retained configurations u_i and v_i; crosses violate the perturbation bound $7x + 2y \leq 14$. The solid segment joins the retained pair. The shaded triangle is the convex hull of the fiber; it does not indicate that its fractional points are valid schedules.

Lemma 3 (global equivalence). The constructed target is feasible if and only if the source has a balanced subset. Every feasible target matrix determines such a subset by selecting precisely its v_i rows.

Proof. Direct substitution in Equation 6 gives

$$w \cdot n = (k+1)(m(2k+1) - A) = \sum_{i=1}^{m} b_i. \tag{9}$$

In any feasible target matrix, Lemma 2 bounds each row's auxiliary objective. Their sum attains the sum of these bounds by Equation 9, so every row attains its own bound. Each row is therefore u_i or v_i.

Let S contain the indices of the v_i rows. The first and third column sums give

$$\sum_{i \in S} a_i = \frac{A}{2}, \quad |S|(k+1) - \frac{A}{2} = \frac{m}{2}(k+1) - \frac{A}{2}. \tag{10}$$

Consequently $|S| = \frac{m}{2}$. This conclusion holds for every valid target witness, without an optimality assumption or a restriction on the solver's choice.

Conversely, place v_i on machines in a balanced subset S and u_i elsewhere. Lemma 2 gives row feasibility. The first and third column sums follow from the two balanced-subset equations. The second column sum is

$$\sum_{i \notin S} (2k + 1 - a_i) = \frac{m}{2}(2k + 1) - \frac{A}{2}, \tag{11}$$

as required. The special empty and odd-total instances preserve their decisions by construction. □

The recovery algorithm scans the count matrix and compares row i with the two explicit configurations in Equation 4. It returns the indices of the v_i rows and checks their cardinality and sum. Lemma 3 ensures that every valid target witness passes these checks. For the empty source it returns the empty set. A NO target has no witness requiring recovery.

5. Encoding size and running time

Let L be the source bit length and let $\ell = 1 + \lceil \log_2(k + 1) \rceil$. The auxiliary processing times are $O(k^3)$, local capacities are $O(k^4)$, and multiplicities are $O(mk)$. Thus the product deadline has $O(m\ell)$ bits, as does each scaled processing time. The total output length is

$$O(m^2\ell + m \log m) = O(L^3). \tag{12}$$

Taking a product increases numerical magnitude substantially but does not create exponentially many output bits.

The forward algorithm uses $O(m)$ multiplications and exact divisions on integers of at most $O(m\ell)$ bits. Schoolbook arithmetic gives the conservative bound $O(L^5)$ bit operations. Integer serialization is polynomial as well. The algorithm expands neither jobs nor integer configuration sets, and uses no solver or randomness.

Recovery uses a scan, comparisons and integer sums. A conservative bound is $O(B^2)$ bit operations, where B is the combined encoded length of the source and supplied witness. Its output uses $O(m \log m)$ bits.

The target belongs to NP. Each allocation entry is at most its type's multiplicity, so the count witness has polynomial bit length. The column sums and row loads can be checked by polynomial-time integer arithmetic. Source NP-hardness and Lemma 3 now prove Theorem 1.

6. Scope of the result

The three-type selector supplies both a decision reduction and universal witness recovery. Its large binary multiplicities and processing times are within the permitted model; the argument establishes ordinary NP-completeness. It does not establish strong NP-hardness, an approximation lower bound, or a new result for uniform machines.

Bibliography

[1] M. Koutecký and J. Zink, "Complexity of scheduling few types of jobs on related and unrelated machines," *Journal of Scheduling*, vol. 28, pp. 139–156, 2025, doi: 10.1007/s10951-024-00827-8.

[2] S. T. McCormick, S. R. Smallwood, and F. C. R. Spieksma, "A polynomial algorithm for multiprocessor scheduling with two job lengths," *Mathematics of Operations Research*, vol. 26, no. 1, pp. 31–49, 2001, doi: 10.1287/moor.26.1.31.10590.

A. Verification and reproducibility

The proof establishes the all-input statement. Finite checks separately test the implementation and its recovered witnesses. The additional verification suite covers 327 source inputs: 94 YES and 233 NO. It enumerates all 521 target count witnesses for these inputs, recovers each source subset, and makes 848 exact Z3 queries, including the final UNSAT query per instance. The scope is sorted lists of length two and four over $1, \ldots, 7$, length six over $1, \ldots, 4$, the empty source, two permutations, and two 51-digit inputs. The prepared eight-input suite also passes, with eight recovery calls; its coverage overlaps the additional suite.

The additional checker uses valid bounds computed from the actual target rows. For any positive vector w, define

$$q_i = \min_j \left(\frac{P_{ij}}{w_j} \right), \quad d_i = \left\lfloor \frac{T}{q_i} \right\rfloor. \tag{13}$$

Every feasible row satisfies $w \cdot c_i \le d_i$. If $w \cdot n = \sum_i d_i$, each row attains its bound. Adding these inequalities and implied equalities preserves every target witness. A separate finite audit checks 221632 feasible configuration/bound pairs. The original constraints remain in the solver.

A separate reviewer directly enumerated 321410 local feasible configurations over 210 pairs $1 \le a \le k \le 20$, and checked five actual-output instances with all five of their target witnesses. Two additional canonical witnesses for a 5001-digit input exercised arbitrary-size serialization. These checks are implementation evidence and do not replace expert assessment of the theorem.

The implementation requires Python 3.11 or later. Verification additionally requires Z3; the recorded runs used Z3 5.1.0. The final reduction itself uses only the Python standard library. The following commands run from the campaign's `work/` directory:

```
python3 algorithm.py <<'JSON'
{"items":[1,2,3,4]}
JSON
python3 algorithm.py --extract <<'JSON'
{"source":{"items":[1,2,3,4]},
 "target_solution":[[1,0,4],[0,7,0],[0,6,0],[4,0,1]]}
JSON
python3 check.py --candidate algorithm.py
python3 verify.py --candidate algorithm.py
python3 ../reviews/affine-selector/check.py
python3 figures/selector.py
typst compile manuscript.typ manuscript.pdf
```

The example recovery returns the zero-based indices `[0,3]`. Checks invoke the two candidate modes in fresh subprocesses and do not import the candidate into their decision oracles. The completed logs, actual target instances and recovered subsets are in `../rounds/009/`. An original large NO query was interrupted when introducing the valid objective bounds; its unknown status and completed prefix are preserved separately. The final equivalent encoding completed the full scope. No timeout or unknown-to-NO conversion was used.

Primary-literature searches were checked on 16 September 2026 against the four-type theorem in [1] and its arXiv record, earlier two-length work, and related integer-cone and scheduling results. No matching later three-type hardness theorem was located. This search is not exhaustive: unindexed work, alternative terminology and inaccessible full text remain coverage limits. In particular, only the publisher's abstract of Sun's 2026 rejection-scheduling paper (DOI 10.1007/s10878-026-01455-y) was available. The full search and independent review are recorded in `../reviews/affine-selector/review.md`. This manuscript has not received human peer review and has not been submitted or published.